5. To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 100

typedef struct {
    char value[MAX_TOKEN_LENGTH];
    int quoted;
} Token;

/* Parse input and handle single quotes */
int parseInput(char *input, Token tokens[]) {

    int count = 0;
    int i = 0;

    while (input[i] != '\0' && count < MAX_TOKENS) {

        /* Skip whitespace outside quotes */
        while (input[i] == ' ' || input[i] == '\t') {
            i++;
        }

        if (input[i] == '\0')
            break;

        int j = 0;
        int quoted = 0;

        /* Handle single-quoted string */
        if (input[i] == '\'') {

            quoted = 1;
            i++;

            while (input[i] != '\0' && input[i] != '\'') {

                if (j < MAX_TOKEN_LENGTH - 1) {
                    tokens[count].value[j++] = input[i];
                }

                i++;
            }

            /* Missing closing quote */
            if (input[i] == '\0') {
                printf("Error: Missing closing single quote.\n");
                return -1;
            }
```

```
  GNU nano 8.7.1
            /* Missing closing quote */
            if (input[i] == '\0') {
                printf("Error: Missing closing single quote.\n");
                return -1;
            }

            /* Skip closing quote */
            i++;
        }

        /* Handle normal unquoted text */
        else {

            while (input[i] != '\0' &&
                   input[i] != ' ' &&
                   input[i] != '\t') {

                if (input[i] == '\'') {
                    printf("Error: Unexpected single quote.\n");
                    return -1;
                }

                if (j < MAX_TOKEN_LENGTH - 1) {
                    tokens[count].value[j++] = input[i];
                }

                i++;
            }
        }

        tokens[count].value[j] = '\0';
        tokens[count].quoted = quoted;

        count++;
    }

    return count;
}

/* Display parsing result */
void displayTokens(Token tokens[], int count) {

    printf("\nParsing Result:\n");
    printf("-----------------------------------------\n");

    for (int i = 0; i < count; i++) {

        printf("Token %d : %s", i + 1, tokens[i].value);

        if (tokens[i].quoted)
            printf("    [SINGLE QUOTED]");
```

```
  GNU nano 8.7.1
/* Display parsing result */
void displayTokens(Token tokens[], int count) {

    printf("\nParsing Result:\n");
    printf("----------------------------------------\n");

    for (int i = 0; i < count; i++) {

        printf("Token %d : %s", i + 1, tokens[i].value);

        if (tokens[i].quoted)
            printf("    [SINGLE QUOTED]");

        else
            printf("    [NORMAL]");

        printf("\n");
    }

    printf("----------------------------------------\n");
}

/* Validate parsing result */
void validateTokens(Token tokens[], int count) {

    if (count == 0) {
        printf("No tokens found.\n");
        return;
    }

    printf("Parsing result is valid.\n");
}

int main() {

    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("Single Quote Parser\n");
    printf("-------------------\n");

    printf("Enter input: ");

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    int count = parseInput(input, tokens);

    if (count == -1) {
        return 1;
```

```
    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    int count = parseInput(input, tokens);

    if (count == -1) {
        return 1;
    }

    displayTokens(tokens, count);

    validateTokens(tokens, count);

    return 0;
}
```

Result:

```
eekshitha_20@LAPTOP-EPSMTP75:~$ nano skill5a.c
eekshitha_20@LAPTOP-EPSMTP75:~$ gcc skill5a.c -o s5a
eekshitha_20@LAPTOP-EPSMTP75:~$ ./s5a
Single Quote Parser
-----------------------------------
Enter input: echo 'Hello World'

Parsing Result:
-----------------------------------
Token 1: echo [NORMAL]
Token 2: Hello World [SINGLE QUOTED]
-----------------------------------
Parsing result is valid.
eekshitha_20@LAPTOP-EPSMTP75:~$ echo '$USER'
$USER
eekshitha_20@LAPTOP-EPSMTP75:~$ echo 'Hello World
> |
```

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Code:



```
GNU nano 8.7.1
                }

                if (j < MAX_TOKEN_LENGTH - 1)
                    tokens[count].value[j++] = input[i];

                i++;
            }
        }

        tokens[count].value[j] = '\0';
        tokens[count].quoted = quoted;

        count++;
    }

    return count;
}

/* Display parsing results */
void displayTokens(Token tokens[], int count) {

    printf("\nParsing Result:\n");
    printf("-----------------------------------------\n");

    for (int i = 0; i < count; i++) {

        printf("Token %d : %-20s ", i + 1,
               tokens[i].value);

        if (tokens[i].quoted)
            printf("[DOUBLE QUOTED]\n");
        else
            printf("[NORMAL]\n");
    }

    printf("-----------------------------------------\n");
}

/* Validate output */
void validateOutput(Token tokens[], int count) {

    if (count == 0) {
        printf("No tokens found.\n");
        return;
    }

    printf("Parsing output is valid.\n");
}

int main() {

    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("Double Quote Parser\n");
    printf("-------------------\n");

    printf("Enter command: ");

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    int count = parseInput(input, tokens);

    if (count == -1)
        return 1;

    displayTokens(tokens, count);

    validateOutput(tokens, count);

    return 0;
}
```

Result:

```
ketana_sindhu@KetaZone: ~/
ketana_sindhu@KetaZone:~/SKILL$ nano Skill5b.c
ketana_sindhu@KetaZone:~/SKILL$ gcc Skill5b.c -o s5b
ketana_sindhu@KetaZone:~/SKILL$ ./s5b
Double Quote Parser
-------------------
Enter command: echo "Hello World"

Parsing Result:
-----------------------------------------
Token 1 : echo                  [NORMAL]
Token 2 : Hello World           [DOUBLE QUOTED]
-----------------------------------------
Parsing output is valid.
ketana_sindhu@KetaZone:~/SKILL$ ./s5b
Double Quote Parser
-------------------
Enter command: echo "Hello World from Linux"

Parsing Result:
-----------------------------------------
Token 1 : echo                  [NORMAL]
Token 2 : Hello World from Linux    [DOUBLE QUOTED]
-----------------------------------------
Parsing output is valid.
ketana_sindhu@KetaZone:~/SKILL$ export USER=Ketana
ketana_sindhu@KetaZone:~/SKILL$ echo "Hello $USER"
Hello Ketana
ketana_sindhu@KetaZone:~/SKILL$ ./s5b
Double Quote Parser
-------------------
Enter command: echo "Hello $USER"

Parsing Result:
-----------------------------------------
Token 1 : echo                  [NORMAL]
Token 2 : Hello Ketana          [DOUBLE QUOTED]
-----------------------------------------
Parsing output is valid.
ketana_sindhu@KetaZone:~/SKILL$
```